

Análisis Teórico Extendido: Minimización de Función Objetivo

Kevin Alejandro Torres Perera
Grupo C311

13 de noviembre de 2025

Enlace del proyecto: <https://github.com/KevinTorres01/OptimProject/>

Resumen

Este documento presenta un análisis teórico completo de un problema de optimización sin restricciones. El objetivo es minimizar la función $f(x, y) = 10(y - x^2)^2 + \frac{(x-1)^2}{2} + (y-1)^2$ en el dominio $\mathbb{R}^2$. El análisis incluye: caracterización matemática de la función, cálculo del gradiente y Hessiana, estudio de convexidad, determinación analítica del mínimo global, y descripción detallada de los algoritmos numéricos (Newton-CG y BFGS) que serán empleados para resolver el problema computacionalmente.

Resultado principal: La función posee un único punto estacionario en $(1, 1)$ que corresponde al mínimo global único con valor $f(1, 1) = 0$.

Índice

1. Definición de la Función Objetivo	4
1.1. Dominio	4
1.2. Signo de la Función	4
2. Análisis de Variables	4
2.1. Tipo de Variables	4
3. Continuidad y Diferenciabilidad	5
3.1. Continuidad	5
3.2. Diferenciabilidad	5
4. Gradiente	5
4.1. Cálculo de Derivadas Parciales	5
4.2. Vector Gradiente	6
5. Matriz Hessiana	6
5.1. Cálculo de Segundas Derivadas	6
5.2. Matriz Hessiana Final	6
6. Análisis de Convexidad	6
6.1. Criterio de Definición Positiva	6
6.2. Conclusión sobre Convexidad	7
7. Determinación del Mínimo Teórico	7
7.1. Condición Necesaria de Primer Orden: Puntos Estacionarios	7
7.2. Solución de la Ecuación Cúbica	8
7.3. Punto Estacionario Único	8
7.4. Verificación del Tipo de Punto Estacionario	8
7.5. Verificación del Mínimo Global	9
8. Análisis de Puntos Estacionarios y Óptimo	9
8.1. Resumen de Puntos Estacionarios	9
8.2. Características del Óptimo Global	9
8.3. Análisis en el Rango de Experimentación $[-100, 100]$	9
8.4. Verificación: ¿Minimización o Maximización?	10
9. Descripción de los Algoritmos Utilizados	10
9.1. Método de Newton-CG (Conjugate Gradient)	10
9.1.1. Fundamento Teórico	10
9.1.2. Formulación Matemática	10
9.1.3. Ventajas	10
9.1.4. Implementación con SciPy	11
9.1.5. Razones para la Selección del Algoritmo	11
9.2. Método Quasi-Newton BFGS	12
9.2.1. Fundamento Teórico	12
9.2.2. Formulación Matemática	12
9.2.3. Propiedades	12
9.2.4. Implementación con SciPy	12

9.2.5. Razones para la Selección del Algoritmo	13
9.2.6. Posibles Problemas y Limitaciones	13
9.3. Librerías Externas Utilizadas	14
9.3.1. SciPy - Módulo optimize	14
9.3.2. NumPy	14
9.3.3. Matplotlib	14
9.3.4. Justificación del Uso de Librerías	15
10.Comparación de Resultados	15
10.1. Criterios de Comparación	15
10.2. Variables Experimentales	15
10.3. Hipótesis de Comportamiento Esperado	16
10.4. Análisis del Impacto de Parámetros	16
11.Visualización del Modelo	17
11.1. Gráficas a Generar	17
11.2. Interpretación Geométrica	17
11.3. Posibles Dificultades para los Algoritmos	17
12.Conclusiones del Análisis Teórico	18

1. Definición de la Función Objetivo

La función objetivo a minimizar está definida como:

$$f(x, y) = 10(y - x^2)^2 + \frac{(x - 1)^2}{2} + (y - 1)^2 \quad (1)$$

Nota importante: Se ha verificado que el problema es de **minimización** (no maximización). Todos los términos de la función son no negativos, y el análisis confirma que existe un mínimo global en $(1, 1)$ con valor 0. Si fuera necesario resolver un problema de maximización, se trabajaría con $-f(x, y)$.

1.1. Dominio

El dominio de la función es:

$$\text{Dom}(f) = \mathbb{R}^2 \quad (2)$$

La función está definida para todos los pares ordenados $(x, y) \in \mathbb{R}^2$, ya que todas las operaciones involucradas (potencias, sumas, productos) son válidas para cualquier número real.

1.2. Signo de la Función

Analizando cada término:

- $10(y - x^2)^2 \geq 0$ (cuadrado multiplicado por constante positiva)
- $\frac{(x-1)^2}{2} \geq 0$ (cuadrado multiplicado por constante positiva)
- $(y - 1)^2 \geq 0$ (cuadrado)

Por lo tanto: $f(x, y) \geq 0$ para todo $(x, y) \in \mathbb{R}^2$

La función alcanza el valor $f(x, y) = 0$ si y solo si simultáneamente:

- $y - x^2 = 0$
- $x - 1 = 0$
- $y - 1 = 0$

De las dos últimas ecuaciones: $x = 1$ y $y = 1$. Verificando la primera: $1 - 1^2 = 0$ ✓

Por lo tanto, $f(x, y) = 0$ únicamente en el punto $(1, 1)$ y $f(x, y) > 0$ en cualquier otro punto.

2. Análisis de Variables

2.1. Tipo de Variables

- **Variable** x : Continua, $x \in \mathbb{R}$
- **Variable** y : Continua, $y \in \mathbb{R}$

Ambas variables son continuas y no presentan restricciones de dominio, lo que permite aplicar métodos de optimización basados en cálculo diferencial.

3. Continuidad y Diferenciabilidad

3.1. Continuidad

La función $f(x, y)$ es una composición de funciones elementales:

- Polinomios: x^2 , $(x - 1)^2$, $(y - 1)^2$
- Combinaciones lineales y cuadráticas

Cada componente es continua en $\mathbb{R}^2$, por lo tanto:

$$f \in C^0(\mathbb{R}^2) \quad (3)$$

La función es **continua en todo su dominio**.

3.2. Diferenciabilidad

Como f es un polinomio en las variables x e y , todas sus derivadas parciales existen y son continuas en $\mathbb{R}^2$.

Por lo tanto:

$$f \in C^\infty(\mathbb{R}^2) \quad (4)$$

La función es **infinitamente diferenciable** (suave) en todo su dominio.

4. Gradiente

El gradiente de la función es el vector de derivadas parciales:

$$\nabla f(x, y) = \begin{bmatrix} \frac{\partial f}{\partial x} \\ \frac{\partial f}{\partial y} \end{bmatrix} \quad (5)$$

4.1. Cálculo de Derivadas Parciales

Derivada parcial respecto a x :

$$\frac{\partial f}{\partial x} = \frac{\partial}{\partial x} \left[10(y - x^2)^2 + \frac{(x - 1)^2}{2} + (y - 1)^2 \right] \quad (6)$$

$$= 10 \cdot 2(y - x^2) \cdot (-2x) + \frac{2(x - 1)}{2} + 0 \quad (7)$$

$$= -40x(y - x^2) + (x - 1) \quad (8)$$

$$= -40xy + 40x^3 + x - 1 \quad (9)$$

Derivada parcial respecto a y :

$$\frac{\partial f}{\partial y} = \frac{\partial}{\partial y} \left[10(y - x^2)^2 + \frac{(x - 1)^2}{2} + (y - 1)^2 \right] \quad (10)$$

$$= 10 \cdot 2(y - x^2) \cdot 1 + 0 + 2(y - 1) \quad (11)$$

$$= 20(y - x^2) + 2(y - 1) \quad (12)$$

$$= 20y - 20x^2 + 2y - 2 \quad (13)$$

$$= 22y - 20x^2 - 2 \quad (14)$$

4.2. Vector Gradiente

$$\nabla f(x, y) = \begin{bmatrix} 40x^3 - 40xy + x - 1 \\ 22y - 20x^2 - 2 \end{bmatrix} \quad (15)$$

5. Matriz Hessiana

La matriz Hessiana contiene las derivadas parciales de segundo orden:

$$H(x, y) = \begin{bmatrix} \frac{\partial^2 f}{\partial x^2} & \frac{\partial^2 f}{\partial x \partial y} \\ \frac{\partial^2 f}{\partial y \partial x} & \frac{\partial^2 f}{\partial y^2} \end{bmatrix} \quad (16)$$

5.1. Cálculo de Segundas Derivadas

Segunda derivada respecto a x :

$$\frac{\partial^2 f}{\partial x^2} = \frac{\partial}{\partial x}(40x^3 - 40xy + x - 1) = 120x^2 - 40y + 1 \quad (17)$$

Derivada mixta:

$$\frac{\partial^2 f}{\partial x \partial y} = \frac{\partial}{\partial y}(40x^3 - 40xy + x - 1) = -40x \quad (18)$$

Por el teorema de Schwarz (las derivadas cruzadas son iguales para funciones C^2):

$$\frac{\partial^2 f}{\partial y \partial x} = -40x \quad (19)$$

Segunda derivada respecto a y :

$$\frac{\partial^2 f}{\partial y^2} = \frac{\partial}{\partial y}(22y - 20x^2 - 2) = 22 \quad (20)$$

5.2. Matriz Hessiana Final

$$H(x, y) = \begin{bmatrix} 120x^2 - 40y + 1 & -40x \\ -40x & 22 \end{bmatrix} \quad (21)$$

6. Análisis de Convexidad

6.1. Criterio de Definición Positiva

Una función es estrictamente convexa si su Hessiana es definida positiva en todo su dominio.

Para que una matriz 2×2 sea definida positiva, debe cumplir:

1. $H_{11} > 0$
2. $\det(H) > 0$

Primer criterio:

$$H_{11} = 120x^2 - 40y + 1 \quad (22)$$

Este término puede ser negativo para ciertos valores de (x, y) . Por ejemplo, en $(0, 1)$:

$$H_{11}(0, 1) = 0 - 40 + 1 = -39 < 0 \quad (23)$$

Determinante:

$$\det(H) = (120x^2 - 40y + 1)(22) - (-40x)^2 \quad (24)$$

$$= 2640x^2 - 880y + 22 - 1600x^2 \quad (25)$$

$$= 1040x^2 - 880y + 22 \quad (26)$$

En el punto $(0, 1)$:

$$\det(H)(0, 1) = 0 - 880 + 22 = -858 < 0 \quad (27)$$

6.2. Conclusión sobre Convexidad

La función NO es convexa en todo $\mathbb{R}^2$, ya que la Hessiana no es definida positiva en todos los puntos del dominio.

Sin embargo, esto no impide que la función tenga un mínimo global único. La no convexidad puede crear desafíos para algunos algoritmos de optimización, especialmente en regiones alejadas del óptimo, pero los métodos basados en Newton y quasi-Newton pueden manejar esta situación al utilizar información de curvatura local.

7. Determinación del Mínimo Teórico

7.1. Condición Necesaria de Primer Orden: Puntos Estacionarios

Como este es un problema de optimización sin restricciones, los candidatos a óptimo son los puntos estacionarios, aquellos donde el gradiente se anula. Los puntos estacionarios satisfacen $\nabla f(x, y) = \mathbf{0}$:

$$\begin{cases} 40x^3 - 40xy + x - 1 = 0 \\ 22y - 20x^2 - 2 = 0 \end{cases} \quad (28)$$

De la segunda ecuación:

$$22y = 20x^2 + 2 \quad (29)$$

$$y = \frac{20x^2 + 2}{22} = \frac{10x^2 + 1}{11} \quad (30)$$

Sustituyendo en la primera ecuación:

$$40x^3 - 40x \left(\frac{10x^2 + 1}{11} \right) + x - 1 = 0 \quad (31)$$

$$40x^3 - \frac{400x^3 + 40x}{11} + x - 1 = 0 \quad (32)$$

Multiplicando por 11:

$$440x^3 - 400x^3 - 40x + 11x - 11 = 0 \quad (33)$$

$$40x^3 - 29x - 11 = 0 \quad (34)$$

7.2. Solución de la Ecuación Cúbica

Probando $x = 1$:

$$40(1)^3 - 29(1) - 11 = 40 - 29 - 11 = 0 \quad (35)$$

Por lo tanto, $x = 1$ es una raíz. Factorizando:

$$40x^3 - 29x - 11 = (x - 1)(40x^2 + 40x + 11) \quad (36)$$

Para las otras raíces:

$$40x^2 + 40x + 11 = 0 \quad (37)$$

Discriminante:

$$\Delta = 40^2 - 4(40)(11) = 1600 - 1760 = -160 < 0 \quad (38)$$

No hay raíces reales adicionales.

7.3. Punto Estacionario Único

Con $x = 1$:

$$y = \frac{10(1)^2 + 1}{11} = \frac{11}{11} = 1 \quad (39)$$

Único punto estacionario: $(x^*, y^*) = (1, 1)$

7.4. Verificación del Tipo de Punto Estacionario

Para determinar si el punto estacionario es un mínimo, máximo o punto silla, evaluamos la Hessiana en $(1, 1)$:

$$H(1, 1) = \begin{bmatrix} 120(1)^2 - 40(1) + 1 & -40(1) \\ -40(1) & 22 \end{bmatrix} = \begin{bmatrix} 81 & -40 \\ -40 & 22 \end{bmatrix} \quad (40)$$

Criterio de definición positiva:

$$1. H_{11} = 81 > 0 \checkmark$$

$$2. \det(H) = 81 \cdot 22 - (-40)^2 = 1782 - 1600 = 182 > 0 \checkmark$$

La Hessiana es **definida positiva** en el punto estacionario, lo que confirma por el criterio de la segunda derivada que $(1, 1)$ es un **mínimo local estricto**.

7.5. Verificación del Mínimo Global

Como $f(x, y) \geq 0$ para todo $(x, y) \in \mathbb{R}^2$ (demostrado en la sección 1.2) y $f(1, 1) = 0$, se concluye:

El punto $(1, 1)$ es el mínimo global único con valor $f(1, 1) = 0$.

Además, como $f(x, y) \rightarrow +\infty$ cuando $\|(x, y)\| \rightarrow \infty$ (la función crece sin límite al alejarse del origen debido a los términos cuadráticos y cúbicos), se garantiza que no existen otros mínimos locales en $\mathbb{R}^2$.

8. Análisis de Puntos Estacionarios y Óptimo

8.1. Resumen de Puntos Estacionarios

- **Único punto estacionario:** $(1, 1)$
- **Tipo:** Mínimo local estricto (y también mínimo global único)
- **Valor de la función:** $f(1, 1) = 0$

8.2. Características del Óptimo Global

1. **Unicidad:** El óptimo global es único en todo $\mathbb{R}^2$
2. **Globalidad:** Es mínimo global, no solo local
3. **Cota inferior:** $f(x, y) = 0$ es el valor mínimo absoluto alcanzable
4. **Comportamiento asintótico:** $\lim_{\|(x, y)\| \rightarrow \infty} f(x, y) = +\infty$

8.3. Análisis en el Rango de Experimentación $[-100, 100]$

En el dominio de experimentación $[-100, 100] \times [-100, 100]$:

1. **Mínimo global:** Se encuentra en $(1, 1)$ con $f(1, 1) = 0$
2. **Valores en las esquinas del dominio:**
 - $f(-100, -100) = 10(-100 - 10000)^2 + \frac{(-101)^2}{2} + (-101)^2 \approx 1,02 \times 10^9$
 - $f(100, 100) = 10(100 - 10000)^2 + \frac{(99)^2}{2} + (99)^2 \approx 9,80 \times 10^8$
3. **Máximo en el dominio:** El valor máximo se alcanza aproximadamente en las esquinas, siendo del orden de 10^9
4. **Óptimos locales:** No existen otros óptimos locales en este rango. La función es unimodal (un solo mínimo)
5. **Zonas complicadas para algoritmos:**
 - **Valle parabólico:** La función tiene un valle pronunciado a lo largo de la curva $y = x^2$, donde el término dominante $10(y - x^2)^2$ es pequeño
 - **Regiones alejadas de $(1, 1)$:** En puntos con $|x| > 10$ o $|y| > 10$, el gradiente puede ser muy grande, requiriendo pasos pequeños

- **Cerca del eje y negativo:** Para $x \approx 0$ y $y < 0$, la curvatura cambia significativamente

6. **Condicionamiento:** El número de condición de la Hessiana varía en el dominio. En $(1, 1)$ la matriz está bien condicionada, pero en regiones alejadas puede empeorar.

8.4. Verificación: ¿Minimización o Maximización?

Nota importante: La función original dada es para **minimización**. Verificando:

- Todos los términos son no negativos
- El valor mínimo es 0 en $(1, 1)$
- La función crece indefinidamente al alejarse del mínimo

Si se requiriera maximización, se trabajaría con $-f(x, y)$, pero según el enunciado del problema, se busca **minimizar** $f(x, y)$.

9. Descripción de los Algoritmos Utilizados

9.1. Método de Newton-CG (Conjugate Gradient)

9.1.1. Fundamento Teórico

El método de Newton-CG es una variante del método de Newton que utiliza el método del gradiente conjugado para resolver el sistema lineal asociado en cada iteración. Es especialmente eficiente para problemas de gran escala.

9.1.2. Formulación Matemática

En cada iteración k , el método de Newton clásico resuelve:

$$H(x_k) \cdot d_k = -\nabla f(x_k) \quad (41)$$

donde d_k es la dirección de búsqueda y $H(x_k)$ es la matriz Hessiana.

El método Newton-CG resuelve este sistema de forma **iterativa** usando gradiente conjugado en lugar de invertir directamente la Hessiana, lo que reduce el costo computacional.

La actualización es:

$$x_{k+1} = x_k + \alpha_k d_k \quad (42)$$

donde α_k es el tamaño de paso determinado por búsqueda lineal.

9.1.3. Ventajas

- **Eficiencia computacional:** No requiere invertir la Hessiana explícitamente
- **Convergencia cuadrática:** Similar al método de Newton cuando la Hessiana es definida positiva
- **Adaptabilidad:** Funciona bien incluso cuando la Hessiana no es definida positiva

9.1.4. Implementación con SciPy

Para este proyecto, se utiliza la función `scipy.optimize.minimize` de la librería SciPy (versión 1.5.0 o superior) con el método 'Newton-CG'.

Documentación oficial: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize.html>

Método específico: `method='Newton-CG'`

Descripción según documentación de SciPy:

- Implementa el algoritmo de Newton truncado (Truncated Newton)
- Utiliza el método del gradiente conjugado (CG) para resolver el sistema lineal en cada iteración
- Requiere la función objetivo y su gradiente (Jacobiano)
- La Hessiana puede proporcionarse explícitamente o calcularse por diferencias finitas

Parámetros utilizados en la implementación:

- `jac`: Gradiente calculado analíticamente mediante `grad_f(x)`
- `options={'maxiter': max_iter, 'disp': False}`: Configuración de iteraciones máximas
- `tol`: Tolerancia para el criterio de convergencia

Referencias bibliográficas:

- Wright, S., & Nocedal, J. (1999). Numerical optimization. Springer Science, 35(67-68), 7.
- Nocedal, J., & Wright, S. J. (2006). Numerical Optimization (2nd ed.). Springer.

9.1.5. Razones para la Selección del Algoritmo

Newton-CG se selecciona para este proyecto por las siguientes razones:

1. Aprovecha información de segundo orden (curvatura) de la función mediante la Hessiana
2. Es robusto para funciones no convexas cuando existe un mínimo bien definido
3. Presenta convergencia cuadrática cerca del óptimo cuando la Hessiana es definida positiva
4. La implementación en SciPy es estable y eficiente
5. Permite comparar el beneficio de usar información de segundo orden vs. métodos que solo usan primer orden

9.2. Método Quasi-Newton BFGS

9.2.1. Fundamento Teórico

BFGS (Broyden-Fletcher-Goldfarb-Shanno) es un método quasi-Newton que **aproxima** la matriz Hessiana (o su inversa) mediante actualizaciones sucesivas, evitando el cálculo directo de segundas derivadas.

9.2.2. Formulación Matemática

El método mantiene una aproximación B_k de la Hessiana (o H_k de su inversa) que se actualiza en cada iteración mediante la fórmula de BFGS:

$$B_{k+1} = B_k + \frac{y_k y_k^T}{y_k^T s_k} - \frac{B_k s_k s_k^T B_k}{s_k^T B_k s_k} \quad (43)$$

donde:

- $s_k = x_{k+1} - x_k$ (cambio en posición)
- $y_k = \nabla f(x_{k+1}) - \nabla f(x_k)$ (cambio en gradiente)

La dirección de búsqueda se calcula como:

$$d_k = -H_k \nabla f(x_k) \quad (44)$$

9.2.3. Propiedades

- **Convergencia superlineal:** Más rápida que métodos de primer orden, aunque no cuadrática como Newton
- **Sin segundas derivadas:** Solo requiere evaluar la función y su gradiente
- **Memoria moderada:** Almacena una matriz $n \times n$ (para L-BFGS existe variante con memoria limitada)

9.2.4. Implementación con SciPy

Para este proyecto, se utiliza la función `scipy.optimize.minimize` de la librería SciPy (versión 1.5.0 o superior) con el método 'BFGS'.

Documentación oficial: <https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.minimize.html>

Método específico: `method='BFGS'`

Descripción según documentación de SciPy:

- Implementa el algoritmo quasi-Newton BFGS
- Aproxima la matriz Hessiana inversa mediante actualizaciones de rango 2
- Requiere la función objetivo y preferiblemente su gradiente
- Si no se proporciona el gradiente, lo aproxima por diferencias finitas

Parámetros utilizados en la implementación:

- `jac`: Gradiente calculado analíticamente mediante `grad.f(x)`
- `options={'maxiter': max_iter, 'disp': False}`: Configuración de iteraciones máximas
- `tol`: Tolerancia para el criterio de convergencia

Referencias bibliográficas:

- Broyden, C. G. (1970). The convergence of a class of double-rank minimization algorithms. *IMA Journal of Applied Mathematics*, 6(1), 76-90.
- Fletcher, R. (1970). A new approach to variable metric algorithms. *The Computer Journal*, 13(3), 317-322.
- Goldfarb, D. (1970). A family of variable-metric methods derived by variational means. *Mathematics of Computation*, 24(109), 23-26.
- Shanno, D. F. (1970). Conditioning of quasi-Newton methods for function minimization. *Mathematics of Computation*, 24(111), 647-656.
- Nocedal, J., & Wright, S. J. (2006). *Numerical Optimization* (2nd ed.). Springer.

9.2.5. Razones para la Selección del Algoritmo

BFGS se selecciona para este proyecto por las siguientes razones:

1. Es uno de los métodos quasi-Newton más eficaces y ampliamente utilizados en la práctica
2. No requiere cálculo de la Hessiana, solo necesita el gradiente
3. Presenta un buen balance entre velocidad de convergencia y costo computacional por iteración
4. Es robusto para una amplia variedad de problemas de optimización no lineal
5. Permite comparar su rendimiento con Newton-CG para evaluar la diferencia entre usar información Hessiana exacta vs. aproximada

9.2.6. Posibles Problemas y Limitaciones

Según la literatura y documentación de SciPy, BFGS puede presentar dificultades en:

- **Funciones mal condicionadas:** Cuando la Hessiana tiene un número de condición muy grande
- **Valles estrechos:** En regiones donde la curvatura varía drásticamente en diferentes direcciones (como el valle parabólico $y = x^2$ en nuestra función)
- **Puntos iniciales alejados:** Si el punto inicial está muy lejos del óptimo, puede requerir muchas iteraciones
- **Hessiana singular:** En puntos donde la Hessiana no es invertible (aunque no aplica en nuestro caso en $(1, 1)$)

9.3. Librerías Externas Utilizadas

9.3.1. SciPy - Módulo optimize

Librería: SciPy (Scientific Python)

Versión requerida: $\geq 1.5.0$

Módulo específico: `scipy.optimize`

Documentación oficial: <https://docs.scipy.org/doc/scipy/reference/optimize.html>

Función principal utilizada:

```
scipy.optimize.minimize(fun, x0, method=None, jac=None,
                        options=None, tol=None)
```

Descripción: Función de optimización de uso general que proporciona una interfaz unificada para múltiples algoritmos de minimización sin restricciones y con restricciones.

Métodos específicos utilizados en este proyecto:

1. `method='Newton-CG'` - Newton truncado con gradiente conjugado
2. `method='BFGS'` - Quasi-Newton BFGS

9.3.2. NumPy

Librería: NumPy (Numerical Python)

Versión requerida: $\geq 1.19.0$

Documentación oficial: <https://numpy.org/doc/>

Funciones utilizadas:

- `np.array()`: Creación de vectores para el gradiente
- `np.linalg.norm()`: Cálculo de la norma euclidiana para medir distancia al óptimo

9.3.3. Matplotlib

Librería: Matplotlib

Versión requerida: $\geq 3.3.0$

Documentación oficial: <https://matplotlib.org/stable/index.html>

Módulos utilizados:

- `matplotlib.pyplot`: Generación de gráficas 2D y 3D
- `mpl_toolkits.mplot3d`: Visualizaciones tridimensionales de la superficie

Funciones principales:

- `plt.figure()`, `plt.contour()`, `plt.plot()`: Gráficas de curvas de nivel y trayectorias
- `ax.plot_surface()`: Visualización 3D de la función objetivo

9.3.4. Justificación del Uso de Librerías

Las librerías se utilizan por las siguientes razones:

1. **SciPy:** Contiene implementaciones altamente optimizadas y validadas de algoritmos de optimización. La función `minimize` proporciona una interfaz consistente y criterios de convergencia robustos basados en investigación académica rigurosa.
2. **NumPy:** Es el estándar en Python para computación numérica eficiente. Proporciona operaciones vectorizadas que mejoran el rendimiento computacional.
3. **Matplotlib:** Es la librería estándar para visualización científica en Python, permitiendo crear gráficas de calidad profesional para análisis de resultados.

Todas estas librerías están ampliamente validadas en la comunidad científica y son de código abierto.

10. Comparación de Resultados

10.1. Criterios de Comparación

Los experimentos compararán ambos métodos en:

1. **Número de iteraciones:** Cantidad de pasos hasta convergencia
2. **Número de evaluaciones de función:** Total de llamadas a $f(x, y)$
3. **Número de evaluaciones de gradiente:** Total de llamadas a $\nabla f(x, y)$
4. **Tiempo de ejecución:** Duración total del proceso
5. **Valor final de la función:** $f(x^*, y^*)$ alcanzado
6. **Distancia al óptimo teórico:** $\|(x^*, y^*) - (1, 1)\|$

10.2. Variables Experimentales

Se analizará el comportamiento variando:

- **Puntos iniciales:** Diferentes posiciones de partida (x_0, y_0)
- **Tolerancias:** Criterios de convergencia más o menos estrictos
- **Máximo de iteraciones:** Límite computacional

10.3. Hipótesis de Comportamiento Esperado

Basándose en la teoría de optimización numérica y las características de cada algoritmo, se esperan observar los siguientes comportamientos:

Newton-CG:

- Menor número de iteraciones debido a su convergencia cuadrática cerca del óptimo
- Mayor costo computacional por iteración (requiere resolver un sistema lineal internamente)
- Mejor precisión final en términos de distancia al óptimo teórico
- Posible sensibilidad en regiones donde la Hessiana no es definida positiva

BFGS:

- Mayor número de iteraciones (convergencia superlineal, no cuadrática)
- Menor costo por iteración
- Buena precisión final, aunque potencialmente menor que Newton-CG
- Mayor robustez en regiones problemáticas debido a su naturaleza adaptativa

10.4. Análisis del Impacto de Parámetros

Puntos iniciales en el rango $[-100, 100]$:

- **Puntos cercanos a $(1, 1)$:** Ambos métodos deberían converger rápidamente (< 20 iteraciones)
- **Puntos alejados ($|x| > 50$ o $|y| > 50$):** Newton-CG podría tener ventaja al explotar información de curvatura, pero también podría enfrentar problemas si la Hessiana está mal condicionada
- **Puntos en el valle parabólico ($y \approx x^2, x \neq 1$):** Esta es una zona complicada donde el gradiente puede ser pequeño pero el punto no es óptimo. Requiere análisis cuidadoso del comportamiento de cada algoritmo
- **Puntos con $x \approx 0, y < 0$:** Región de alta curvatura que puede ralentizar la convergencia

Tolerancia de convergencia:

- **Tolerancia alta (10^{-4}):** Ambos métodos convergerán rápidamente pero con menor precisión
- **Tolerancia baja (10^{-10}):** Newton-CG debería alcanzar mayor precisión final debido a su convergencia cuadrática

Máximo de iteraciones:

- Este parámetro actuará como límite de seguridad para evitar ejecuciones infinitas
- Se espera que ambos métodos converjan mucho antes de alcanzar el máximo en la mayoría de casos

11. Visualización del Modelo

11.1. Gráficas a Generar

1. **Superficie 3D:** Representación de $f(x, y)$ en el espacio
2. **Curvas de nivel:** Contornos de la función en el plano (x, y)
3. **Trayectorias de optimización:** Caminos seguidos por cada algoritmo desde distintos puntos iniciales
4. **Convergencia:** Evolución del valor de la función vs. iteraciones

11.2. Interpretación Geométrica

La función presenta las siguientes características geométricas:

- **Valle parabólico:** Existe un valle pronunciado a lo largo de la curva $y = x^2$ debido al término dominante $10(y - x^2)^2$. En esta curva, el primer término es cero.
- **Mínimo único:** Localizado en $(1, 1)$ donde $f(1, 1) = 0$
- **Crecimiento rápido:** La función crece rápidamente al alejarse del mínimo, especialmente en direcciones perpendiculares al valle

11.3. Posibles Dificultades para los Algoritmos

Basándose en el análisis teórico y geométrico, se identifican las siguientes zonas potencialmente problemáticas:

1. **Valle estrecho a lo largo de $y = x^2$:**
 - Los algoritmos pueden tener dificultad para “navegar” eficientemente a lo largo del valle
 - Pueden requerir muchas iteraciones para moverse desde un punto en el valle hasta $(1, 1)$
 - El condicionamiento de la Hessiana puede empeorar en esta región
2. **Regiones con $|x| > 10$:**
 - El término $40x^3$ domina el gradiente, creando gradientes muy grandes
 - Puede requerir tamaños de paso muy pequeños en la búsqueda lineal
 - Newton-CG podría enfrentar problemas si la aproximación cuadrática local no es buena
3. **Puntos cercanos al eje y con $x \approx 0$:**
 - La segunda derivada $\frac{\partial^2 f}{\partial x^2} = 120x^2 - 40y + 1$ puede ser pequeña
 - Si y es positivo y x pequeño, la curvatura en dirección x es débil
4. **Convergencia desde puntos muy alejados (cerca de los límites $[-100, 100]$):**

- Valores de función del orden de 10^9
- Gradientes muy grandes que pueden causar problemas numéricos
- Posible necesidad de estrategias de globalización más sofisticadas

12. Conclusiones del Análisis Teórico

Del análisis matemático realizado, se concluye lo siguiente:

1. **Caracterización del problema:** La función objetivo $f(x, y) = 10(y - x^2)^2 + \frac{(x-1)^2}{2} + (y-1)^2$ es un problema de optimización sin restricciones con un **único punto estacionario** en $(1, 1)$ donde alcanza su **mínimo global con valor** $f(1, 1) = 0$.
2. **Propiedades de regularidad:** La función es infinitamente diferenciable (C^∞) en todo $\mathbb{R}^2$, lo que garantiza la existencia de todas las derivadas necesarias para aplicar los métodos seleccionados.
3. **No convexidad:** Aunque la función **no es convexa** en todo su dominio, esto no impide la existencia de un único mínimo global. La Hessiana es definida positiva en el óptimo, asegurando que es un mínimo local estricto que coincide con el mínimo global.
4. **Adecuación de los algoritmos:** Tanto Newton-CG como BFGS son apropiados para este problema:
 - **Newton-CG:** Explota la información de curvatura (Hessiana) para lograr convergencia cuadrática, ideal cerca del óptimo
 - **BFGS:** Aproxima la Hessiana mediante actualizaciones, ofreciendo robustez y eficiencia computacional
5. **Desafíos esperados:** El valle parabólico a lo largo de $y = x^2$ y las regiones alejadas del óptimo (especialmente cerca de los límites $[-100, 100]$) pueden presentar dificultades para ambos algoritmos. Los experimentos cuantificarán estos efectos.
6. **Validación experimental necesaria:** Los experimentos permitirán:
 - Verificar las tasas de convergencia teóricas
 - Identificar limitaciones prácticas de cada método
 - Evaluar la robustez ante diferentes puntos iniciales
 - Medir el impacto del condicionamiento en regiones problemáticas

Este análisis teórico proporciona las bases matemáticas para interpretar los resultados experimentales que se obtendrán mediante las implementaciones computacionales.